Communication Between Containers

Folder structure:

Python Code:

```
import pymysql

# ------------------ DB CONFIG ------------------ #
DB_HOST = "localhost"
DB_USER = "root"
DB_PASSWORD = "ghat123@gmail.com"
DB_NAME = "employee_db"

# ------------------ DB CONNECTION ------------------ #
def connect_db():
    return pymysql.connect(
        host=DB_HOST,
        user=DB_USER,
        password=DB_PASSWORD,
        cursorclass=pymysql.cursors.Cursor
    )

def setup_database():
    conn = connect_db()
    cursor = conn.cursor()

    cursor.execute(f"CREATE DATABASE IF NOT EXISTS {DB_NAME}")
    cursor.execute(f"USE {DB_NAME}")

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS employees (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            email VARCHAR(100),
```

```
        phone VARCHAR(15) NOT NULL,
        salary FLOAT,
        aptitude_score FLOAT NOT NULL
    )
""")

    conn.commit()
    return conn, cursor

# ------------------ ADD EMPLOYEE ------------------ #
def add_employee(cursor, conn):
    print("\n--- Add Employee ---")

    name = input("Enter Name (required): ").strip()
    phone = input("Enter Phone (required): ").strip()
    email = input("Enter Email (optional): ").strip()
    salary = input("Enter Salary (optional): ").strip()
    aptitude = input("Enter Aptitude Score (0-10) (required): ").strip()

    if not name or not phone or not aptitude:
        print("❌ Employee NOT added. Required fields missing.\n")
        return

    try:
        salary = float(salary) if salary else None
        aptitude = float(aptitude)

        if aptitude < 0 or aptitude > 10:
            print("❌ Aptitude score must be between 0 and 10.\n")
            return

        cursor.execute("""
            INSERT INTO employees (name, email, phone, salary, aptitude_score)
            VALUES (%s, %s, %s, %s, %s)
        """, (name, email if email else None, phone, salary, aptitude))

        conn.commit()
        print("✅ Employee added successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ UPDATE EMPLOYEE ------------------ #
def update_employee(cursor, conn):
```

```
    print("\n--- Update Employee ---")
    emp_id = input("Enter Employee ID: ").strip()

    if not emp_id:
        print("❌ ID not provided.\n")
        return

    try:
        cursor.execute("SELECT * FROM employees WHERE id=%s", (emp_id,))
        if not cursor.fetchone():
            print("❌ Employee not found.\n")
            return

        print("""
a. Update Name
b. Update Email
c. Update Phone
d. Update Salary
e. Update Aptitude Score
""")

        choice = input("Choose option: ").lower()

        field_map = {
            'a': 'name',
            'b': 'email',
            'c': 'phone',
            'd': 'salary',
            'e': 'aptitude_score'
        }

        if choice not in field_map:
            print("❌ Invalid choice.\n")
            return

        new_value = input("Enter new value: ").strip()

        if not new_value:
            print("❌ Value cannot be empty.\n")
            return

        if field_map[choice] == 'salary':
            new_value = float(new_value)
        elif field_map[choice] == 'aptitude_score':
```

```
            new_value = float(new_value)
            if new_value < 0 or new_value > 10:
                print("❌ Aptitude must be between 0 and 10.\n")
                return

        query = f"UPDATE employees SET {field_map[choice]}=%s WHERE id=%s"
        cursor.execute(query, (new_value, emp_id))
        conn.commit()

        print("✅ Employee updated successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ VIEW EMPLOYEES ------------------ #
def view_employees(cursor):
    print("\n--- View Employees ---")
    choice = input("Show all? (y/n): ").lower()

    try:
        if choice == 'y':
            cursor.execute("SELECT * FROM employees")
        else:
            ids = input("Enter min_id max_id: ").split()
            if len(ids) != 2:
                print("❌ Invalid input.\n")
                return

            cursor.execute(
                "SELECT * FROM employees WHERE id BETWEEN %s AND %s",
                (ids[0], ids[1])
            )

        rows = cursor.fetchall()

        if not rows:
            print("⚠️ No employees found.\n")
            return

        print("\n--- Employee Records ---")
        for row in rows:
            print(row)
        print()
```

```
    except Exception as e:
        print("❌ Error:", e)

# ------------------ DELETE EMPLOYEE ------------------ #
def delete_employee(cursor, conn):
    print("\n--- Delete Employee ---")
    emp_id = input("Enter Employee ID to delete: ").strip()

    if not emp_id:
        print("❌ ID not provided.\n")
        return

    try:
        cursor.execute("DELETE FROM employees WHERE id=%s", (emp_id,))
        conn.commit()

        if cursor.rowcount == 0:
            print("❌ Employee not found.\n")
        else:
            print("✅ Employee deleted successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ MAIN LOOP ------------------ #
def main():
    try:
        conn, cursor = setup_database()
        print("✅ Database connected successfully!\n")

        while True:
            print("""
===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit
""")

            choice = input("Enter choice (1-5): ").strip()

            if choice == '1':
                add_employee(cursor, conn)
```

```
        elif choice == '2':
            update_employee(cursor, conn)
        elif choice == '3':
            view_employees(cursor)
        elif choice == '4':
            delete_employee(cursor, conn)
        elif choice == '5':
            print("👋 Exiting program...")
            break
        else:
            print("❌ Invalid input.\n")

    cursor.close()
    conn.close()

  except Exception as e:
    print("❌ Database connection failed:", e)

# ------------------ RUN ------------------ #
if __name__ == "__main__":
  main()
```

Dockerfile:

```
FROM python

WORKDIR /app

COPY PyCode.py .

RUN pip install pymysql

CMD ["python", "PyCode.py"]
```

Run the command:

```
p\Python Code> docker build .
D
ion from Dockerfile
42B
docker.io/library/python:latest
ken for registry-1.docker.io
```

```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker images -a

IMAGE                                ID             DISK USAGE   CONTE
mysql:latest                         24e450bbd24f       1.27GB
mywebapp:02                          b90c1a14ca28       2.11GB
routhkiranbabu/webapp-demo:02        b90c1a14ca28       2.11GB
<untagged>                           094d3676b66d       1.63GB
```

```
\Python Code> docker run -it --rm 094d3676b66d
03, "Can't connect to MySQL server on 'localhost' (
```

Change this in python code:

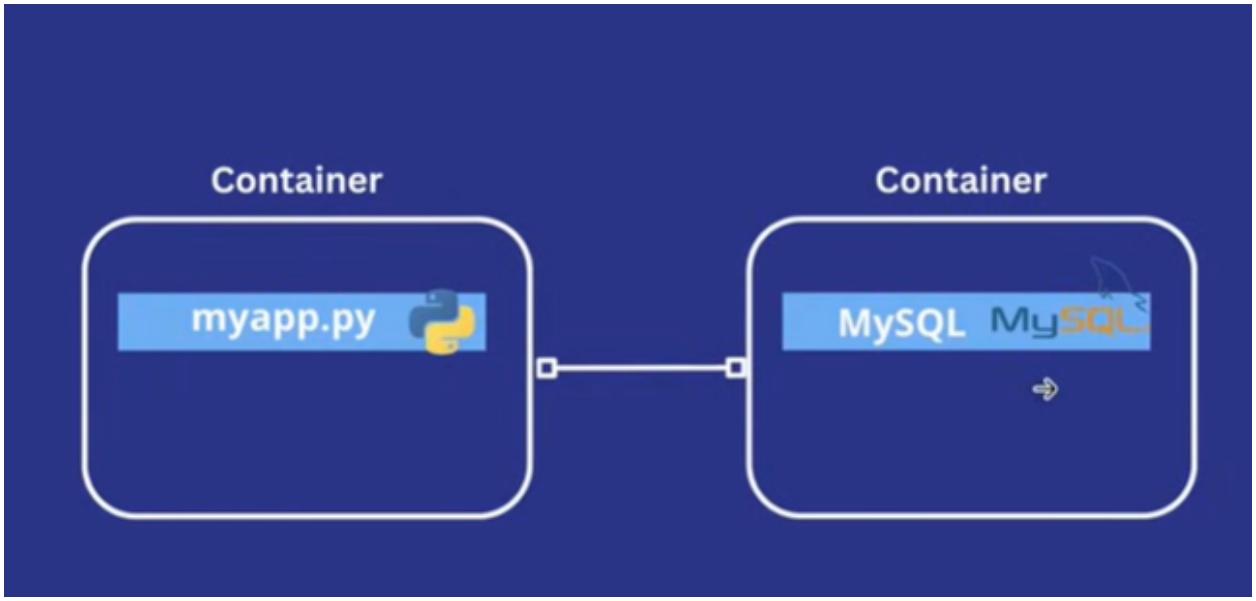
```
kerfile   PyCode.py
Code.py
2
3  # ------------------ DB CONFIG --
4  DB_HOST = "host.docker.internal"
```

```
Python Code> docker rmi 094d3676b66d
b15b50112976f8025294e6030269f5c838a48
Python Code> docker build .

on from Dockerfile
2B
ocker.io/library/python:latest
```

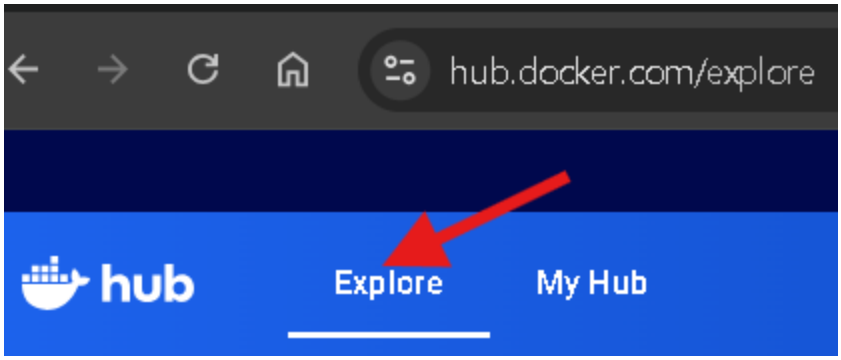
```
Users\kiran\OneDrive\Desktop\Python Code> docker images -a

                                  ID              DISK USAGE   CONTE
latest                            24e450bbd24f       1.27GB
pp:02                             b90c1a14ca28       2.11GB
iranbabu/webapp-demo:02           b90c1a14ca28       2.11GB
ged>                              f9cc710aa4ab       1.63GB
```

```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker run -it --rm f9cc710aa4ab
✅ Database connected successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5):
```

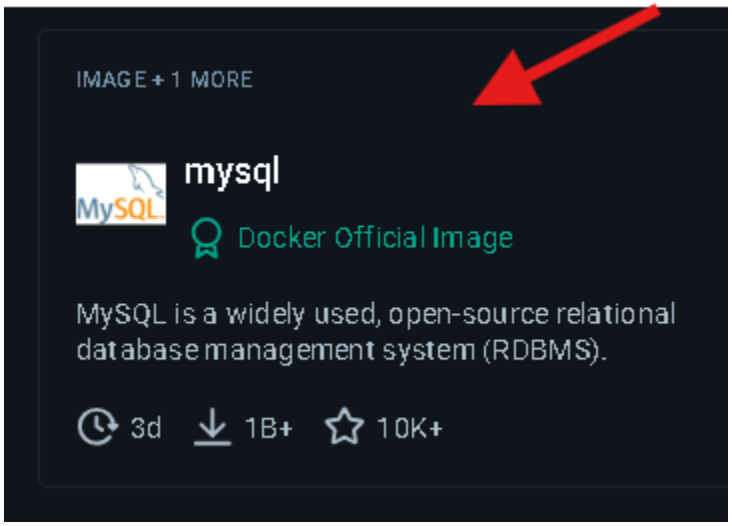
We dont have the mysql container:



To get mysql container:



scroll down




```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker pull mysql
Using default tag: latest
latest: Pulling from library/mysql
Digest: sha256:24e450bbd24f621c71b10404c946cc9ea1cbb0e6e7464b2be2de
Status: Image is up to date for mysql:latest
docker.io/library/mysql:latest
PS C:\Users\kiran\OneDrive\Desktop\Python Code>
```

```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker images

IMAGE                          ID             DISK USAGE   CO
mysql:latest                   24e450bbd24f       1.27GB
mywebapp:02                    b90c1a14ca28       2.11GB
```

-d: detached mode, runs at background. –name: for providing name like mysqldb

```
\Python Code> docker run -d --name mysqldb mysql
7374223451826 40c9940d7778192a0
```

```
\Python Code> docker ps -a
                 CREATED          STATUS                      PORTS     NAMES
ntrypoint.s…"    31 seconds ago   Exited (1) 30 seconds ago             mysqldb
\Python Code>
```

It says it stoped. To check why

```
:\Users\kiran\OneDrive\Desktop\Python Code> docker logs mysqldb
-03-23 14:10:47+00:00 [Note] [Entrypoint]: Entrypoint script fo
-03-23 14:10:47+00:00 [Note] [Entrypoint]: Switching to dedicat
-03-23 14:10:48+00:00 [Note] [Entrypoint]: Entrypoint script fo
-03-23 14:10:48+00:00 [ERROR] [Entrypoint]: Database is uninit
You need to specify one of the following as an environment vari
- MYSQL_ROOT_PASSWORD
- MYSQL_ALLOW_EMPTY_PASSWORD
- MYSQL_RANDOM_ROOT_PASSWORD
```

Visit to know about environment variable:
https://hub.docker.com/_/mysql#environment-variables

Environment Variables

When you start the `mysql` image, you can adjust the configuration of the MySQL instance by passing one or more environment variables on the `docker run` command line. Do note that none of the variables below will have any effect if you start the container with a data directory that already contains a database: any pre-existing database will always be left untouched on container startup.

See also https://dev.mysql.com/doc/refman/5.7/en/environment-variables.html for documentation of environment variables which MySQL itself respects (especially variables like `MYSQL_HOST`, which is known to cause issues when used with this image).

`MYSQL_ROOT_PASSWORD`

This variable is mandatory and specifies the password that will be set for the MySQL `root` superuser account. In the above example, it was set to `my-secret-pw`.

`MYSQL_DATABASE`

This variable is optional and allows you to specify the name of a database to be created on image startup. If a user/password was supplied (see below) then that user will be granted superuser access (corresponding to `GRANT ALL`) to this database.

`MYSQL_USER`, `MYSQL_PASSWORD`

These variables are optional, used in conjunction to create a new user and to set that user's password. This user will be granted superuser permissions (see above) for the database specified by the `MYSQL_DATABASE` variable. Both variables are required for a user to be created.

Do note that there is no need to use this mechanism to create the root superuser, that user gets created by default with the password specified by the `MYSQL_ROOT_PASSWORD` variable.

`MYSQL_ALLOW_EMPTY_PASSWORD`

This is an optional variable. Set to a non-empty value, like `yes`, to allow the container to be started with a blank password for the root user. *NOTE*: Setting this variable to `yes` is not recommended unless you really know what you are doing, since this will leave your MySQL instance completely unprotected, allowing anyone to gain complete superuser access.

`MYSQL_RANDOM_ROOT_PASSWORD`

This is an optional variable. Set to a non-empty value, like `yes`, to generate a random initial password for the root user (using `openssl`). The generated root password will be printed to stdout (`GENERATED ROOT PASSWORD: .....`).

```
docker run -d --env MYSQL_ROOT_PASSWORD="ghat123@gmail.com" --env MYSQL_DATABASE="employee_db" --name mysqldb mysql
```

docker run -d --env MYSQL_ROOT_PASSWORD="ghat123@gmail.com" --env MYSQL_DATABASE="employee_db" --name mysqldb mysql

Shows error: docker: Error response from daemon: Conflict. The container name "/mysqldb" is already in use by container "b359578d9341dbfcd29ab4ec6c018014b6737422345182640c9940d7778192a0". You have to remove (or rename) that container to be able to reuse that name.

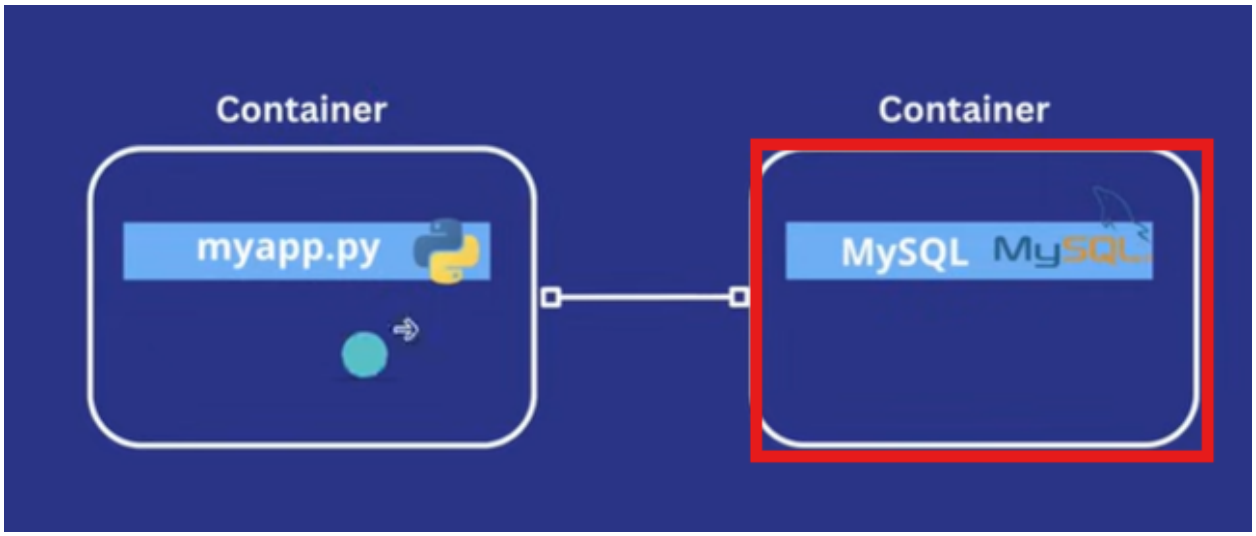
```
\Python Code> docker ps
 CREATED    STATUS     PORTS      NAMES
\Python Code> docker ps -a
                CREATED          STATUS
ntrypoint.s…"   7 minutes ago    Exited (1) 6 minutes ago
\Python Code> docker rm mysqldb
```

After removing run the code again:

```
docker run -d --env MYSQL_ROOT_PASSWORD="ghat123@gmail.com" --env MYSQL_DATABASE="employee_db" --name mysqldb mysql
73401350cdfef1739
```

```
e> docker ps
    CREATED          STATUS          PORTS                    NAMES
."  2 minutes ago    Up 2 minutes    3306/tcp, 33060/tcp      mysqldb
e>
```

Now mysql container is running:



Now need to change the python code to get connect to the mysql container.

```
Python Code> docker inspect mysqldb
```

```
"bridge": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": null,
    "DriverOpts": null,
    "GwPriority": 0,
    "NetworkID": "a430dc9467859897
    "EndpointID": "ff6c8cbed008c2f
    "Gateway": "172.17.0.1",
    "IPAddress": "172.17.0.2",
    "MacAddress": "ea:9a:2b:cd:96:
```

which is the ip address of the container.

Copy the ip address to python code:

```
Dockerfile   PyCode.py
PyCode.py
import pymysql

# ------------------- D
DB_HOST = "172.17.0.2"
DB_USER = "root"
DB_PASSWORD = "ghat123
DB_NAME = "employee db
```

```
BUG CONSOLE   TERMINAL   PORTS   PLAYWRIGHT

"IPAddress": "172.17.0.2",
```

Now lets build:

```
ve\Desktop\Python Code> docker build .
) FINISHED
d definition from Dockerfile
erfile: 142B
data for docker.io/library/python:lates
n:pull token for registry-1.docker.io
```

```
top\Python Code> docker images -a

ID              DISK USAGE   CONTE
24e450bbd24f        1.27GB
b90c1a14ca28        2.11GB
b90c1a14ca28        2.11GB
2c4ce40d1bb1        1.63GB
f9cc710aa4ab        1.63GB
```

```
top\Python Code> docker run -it --rm 2c4ce40d1bb1
'cryptography' package is required for sha256_pass
top\Python Code>
```

So lets add cryptography package. Add this in docker file:

```
Dockerfile   PyCode.py
Dockerfile > ...

RUN pip install pymysql

RUN pip install cryptography
```

Build again:

```
hon Code> docker build .

rom Dockerfile

r.io/library/python:latest
```

```
top\Python Code> docker images -a

ID              DISK USAGE   CONTI
24e450bbd24f        1.27GB
b90c1a14ca28        2.11GB
b90c1a14ca28        2.11GB
2a5af4bc4bb9        1.66GB
2c4ce40d1bb1        1.63GB
f9cc710aa4ab        1.63GB
```

```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker run -it --rm 2a5af4bc4bb9
✅ Database connected successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5):
```

```
Enter choice (1-5): 3

--- View Employees ---
Show all? (y/n): y
⚠️No employees found.


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 1

--- Add Employee ---
Enter Name (required): name1
Enter Phone (required): 2342
Enter Email (optional): df@dfa.com
Enter Salary (optional): 234
Enter Aptitude Score (0-10) (required): 2.234
✅ Employee added successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 1
```

```
Enter choice (1-5): 1

--- Add Employee ---
Enter Name (required): name2
Enter Phone (required): 234234
Enter Email (optional): dfa@dfa.com
Enter Salary (optional): 565
Enter Aptitude Score (0-10) (required): 5.342
✅ Employee added successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 1

--- Add Employee ---
Enter Name (required): name3
Enter Phone (required): 2234
Enter Email (optional): adf@fa.com
Enter Salary (optional): 6456
Enter Aptitude Score (0-10) (required): 7.342
✅ Employee added successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 3
```

```
--- View Employees ---
Show all? (y/n): y

--- Employee Records ---
(1, 'name1', 'df@dfa.com', '2342', 234.0, 2.234)
(2, 'name2', 'dfa@dfa.com', '234234', 565.0, 5.342)
(3, 'name3', 'adf@fa.com', '2234', 6456.0, 7.342)
```

Only one container is running:

```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED
c784acdab674   mysql     "docker-entrypoint.s…"   22 minut
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED
c784acdab674   mysql     "docker-entrypoint.s…"   22 minut
PS C:\Users\kiran\OneDrive\Desktop\Python Code>
```

If I run the code again:

```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker run -it --rm 2a5af4bc4bb9
✅ Database connected successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 3

--- View Employees ---
Show all? (y/n): y

--- Employee Records ---
(1, 'name1', 'df@dfa.com', '2342', 234.0, 2.234)
(2, 'name2', 'dfa@dfa.com', '234234', 565.0, 5.342)
(3, 'name3', 'adf@fa.com', '2234', 6456.0, 7.342)
```

It saved all value.

How to start/stop Containers?

```
\Python Code> docker ps
 CREATED   STATUS    PORTS     NAMES
\Python Code> docker ps -a
                CREATED        STATUS                        PORTS                 NAMES
ntrypoint.s…"   16 hours ago   Exited (255) 58 seconds ago   3306/tcp, 33060/tcp   mysqldb
\Python Code> docker start mysqldb

\Python Code> docker ps -a
                CREATED        STATUS         PORTS                 NAMES
ntrypoint.s…"   16 hours ago   Up 2 seconds   3306/tcp, 33060/tcp   mysqldb
\Python Code> docker stop mysqldb

\Python Code> docker ps -a
                CREATED        STATUS                     PORTS     NAMES
ntrypoint.s…"   16 hours ago   Exited (0) 3 seconds ago             mysqldb
\Python Code>
```

```
top\Python Code> docker start mysqldb

top\Python Code> docker images -a

 ID              DISK USAGE   CONTENT SIZE   EXTRA
 24e450bbd24f       1.27GB          283MB     U
 b90c1a14ca28       2.11GB          491MB
 b90c1a14ca28       2.11GB          491MB
 2a5af4bc4bb9       1.66GB          429MB
 2c4ce40d1bb1       1.63GB          418MB
 f9cc710aa4ab       1.63GB          418MB
top\Python Code> docker run -it --rm 2a5af4bc4bb9
```

```
✅ Database connected successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 3

--- View Employees ---
Show all? (y/n): y

--- Employee Records ---
(1, 'name1', 'df@dfa.com', '2342', 234.0, 2.234)
(2, 'name2', 'dfa@dfa.com', '234234', 565.0, 5.342)
(3, 'name3', 'adf@fa.com', '2234', 6456.0, 7.342)


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit
```

So we can start the container and it shows it still stores the values.